

Tutorial: RAG

Dương Trường Bình

Nguyễn Anh Khôi

Trần Đại Nhân

Dương Đình Thắng

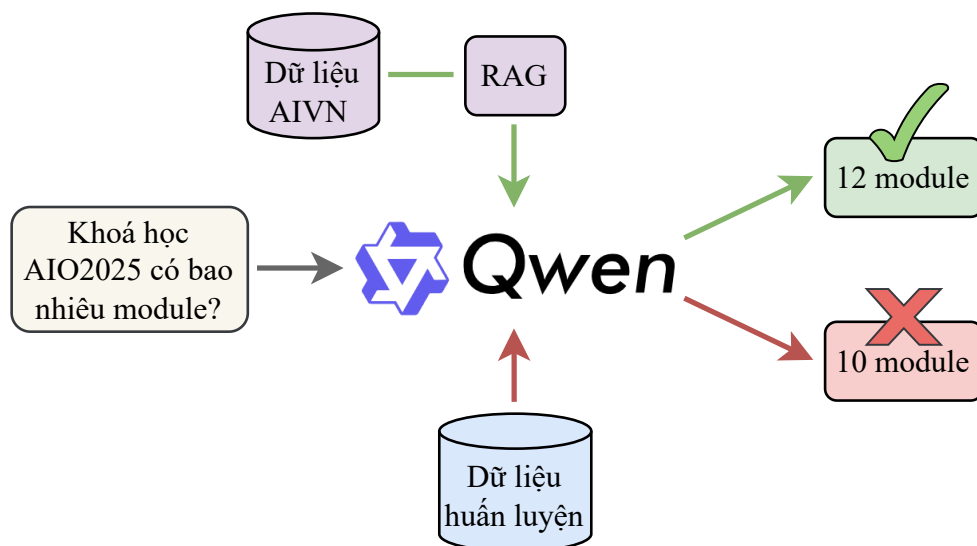
Đinh Quang Vinh

I. Dẫn nhập

Sự bùng nổ của các Large Language Models (LLMs) như ChatGPT, Gemini, Claude hay Llama đã định hình lại lĩnh vực NLP. Tuy nhiên, dù sở hữu khả năng tổng quát và suy luận ấn tượng, các mô hình này vẫn đối mặt với những hạn chế cố hữu: tri thức bị giới hạn tại thời điểm huấn luyện, hiện tượng ảo giác khi gặp các câu hỏi nằm ngoài vùng tri thức, và đặc biệt là sự thiếu hụt kiến thức về dữ liệu riêng tư của doanh nghiệp.

Để giải quyết vấn đề này, kỹ thuật Retrieval Augmented Generation (RAG) đã ra đời. RAG cho phép LLMs tiếp cận nguồn dữ liệu bên ngoài mà không cần trải qua quá trình fine-tuning tốn kém hay phải huấn luyện lại. Để giới thiệu về RAG, bài viết này đi sâu vào kiến trúc và cách triển khai, bao gồm:

1. Phân tích sâu khái niệm, kiến trúc, pipeline cơ bản của RAG.
2. Giới thiệu framework LangChain, công cụ mạnh mẽ cho các ứng dụng sử dụng LLM.
3. Xây dựng hệ thống QA System trên tài liệu PDF học thuật.



Hình 1: Minh hoạ LLM khi có (đường xanh lá) và không sử dụng RAG (đường màu đỏ).

Mục lục

I.	Dẫn nhập	1
II.	Cơ sở lý thuyết về RAG	4
II.1.	Sự chuyển dịch sang In-Context RAG	5
II.2.	Kiến trúc RAG hiện đại	5
III.	Thư viện LangChain	15
III.1.	Giới thiệu	15
III.2.	Các thành phần cốt lõi	15
IV.	Thực hành	19
IV.1.	Chuẩn bị môi trường và cấu trúc dự án	19
IV.2.	Chuẩn bị dữ liệu PDF đầu vào	21
IV.3.	Tiền xử lý văn bản và Chunking	22
IV.4.	Xây dựng Vector Database	23
IV.5.	Khởi tạo LLM và xây dựng RAG Chain	24
IV.6.	Xây dựng giao diện ứng dụng	27
V.	Câu hỏi trắc nghiệm	29
	Phụ lục	33

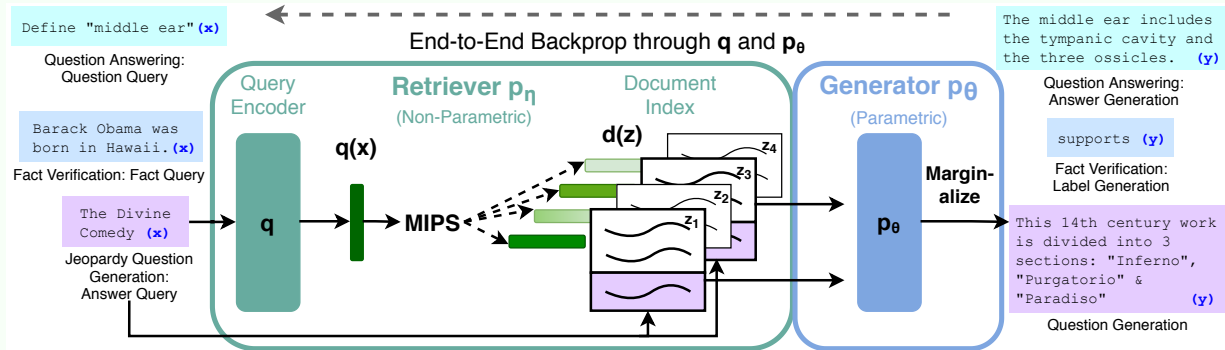
Bảng thuật ngữ

Thuật ngữ	Mô tả
Hallucination	Hiện tượng mô hình sinh ra thông tin sai lệch, bịa đặt hoặc không có thực nhưng với văn phong tự tin.
Knowledge Cutoff	Mốc thời gian giới hạn của dữ liệu huấn luyện, khiến mô hình không biết các sự kiện xảy ra sau đó.
Fine-tuning	Quá trình huấn luyện tiếp mô hình đã pre-trained trên tập dữ liệu chuyên biệt để cập nhật trọng số.
In-Context Learning	Khả năng LLM học và thực hiện tác vụ dựa trên ngữ cảnh hoặc ví dụ được cung cấp trong prompt mà không cần cập nhật tham số.
Vector Embeddings	Biểu diễn dữ liệu (text, image) dưới dạng vector số thực trong không gian n-chiều.
Semantic Search	Tìm kiếm dựa trên sự tương đồng về ý nghĩa thay vì chỉ khớp từ khóa.
Chunking	Kỹ thuật chia nhỏ văn bản dài thành các đoạn ngắn để tối ưu hóa việc mã hóa và phù hợp với giới hạn Context Window.
Context Window	Giới hạn số lượng tokens (đơn vị văn bản) tối đa mà LLM có thể tiếp nhận và xử lý trong một lần prompt.
Grounding	Kỹ thuật “neo” câu trả lời của mô hình vào dữ liệu thực tế được cung cấp để đảm bảo tính xác thực.

II. Cơ sở lý thuyết về RAG

i Nguồn gốc

Khái niệm RAG lần đầu tiên được đề xuất chính thức trong bài báo khoa học “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks” bởi Patrick Lewis và các cộng sự tại Facebook AI Research (FAIR) vào năm 2020^a.



Hình 2: Tổng quan kiến trúc RAG trong bài báo gốc của Patrick Lewis (2020).

Trong công trình này, nhóm tác giả định nghĩa RAG là một mô hình xác suất lai (hybrid probabilistic model) kết hợp giữa hai dạng bộ nhớ nhằm khắc phục nhược điểm của các mô hình Pre-trained Seq2Seq truyền thống:

- **Parametric Memory (Bộ nhớ tham số):** Là tri thức ẩn được lưu trữ trong trọng số của một mô hình sinh chuỗi (Pre-trained Seq2Seq Transformer). Cụ thể trong bài báo, tác giả sử dụng mô hình BART (Bidirectional and Auto-Regressive Transformers) làm Generator.
- **Non-Parametric Memory (Bộ nhớ phi tham số):** Là tri thức tường minh từ bên ngoài, cụ thể là một dense vector index chứa các đoạn văn bản Wikipedia. Thành phần này được truy cập thông qua một bộ Neural Retriever dựa trên kiến trúc Dense Passage Retriever.

Cơ chế hoạt động của RAG gốc cho phép Generator (BART) sử dụng đầu vào kết hợp với các tài liệu ẩn (latent documents) tìm được từ Retriever để sinh văn bản. Điểm đặc biệt là toàn bộ kiến trúc này được fine-tuning end-to-end, cho phép cập nhật trọng số của cả Query Encoder và Generator để tối ưu hóa tác vụ đích.

^aXem chi tiết tại: <https://arxiv.org/abs/2005.11401>

II.1. Sự chuyển dịch sang In-Context RAG

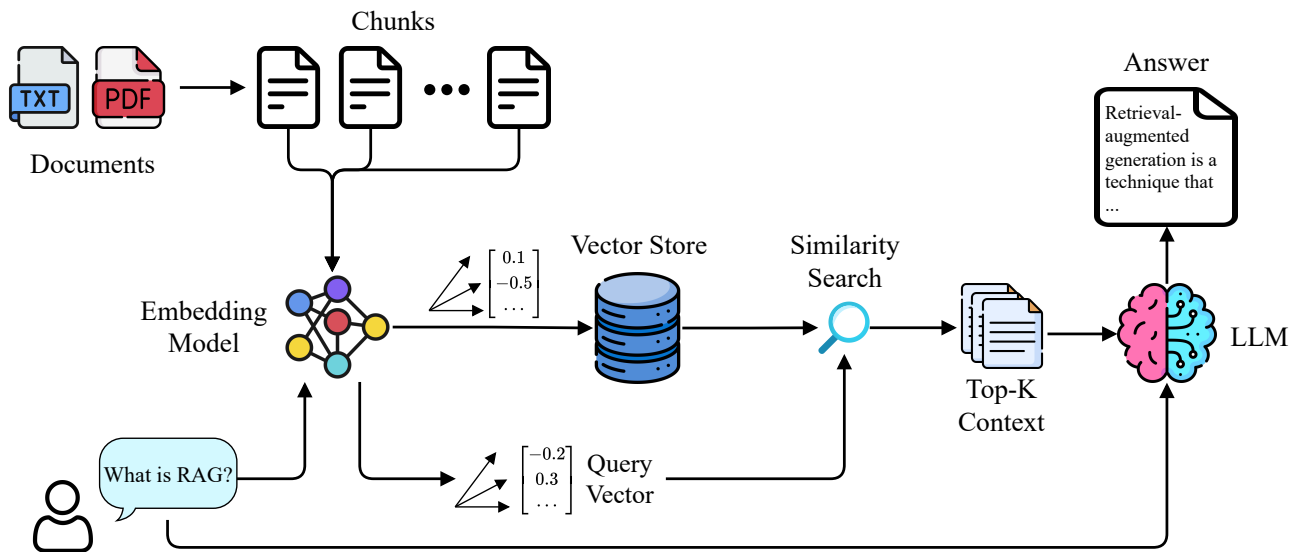
Mặc dù thuật ngữ RAG vẫn được giữ nguyên, tư duy triển khai kỹ thuật này đã thay đổi căn bản cùng với sự phát triển của các LLM:

- **Original RAG (2020):** Là hướng tiếp cận dựa trên fine-tuning. Như đã đề cập ở phần trên, mô hình gốc yêu cầu huấn luyện đồng thời cả bộ truy xuất và mô hình sinh văn bản để chúng học cách phối hợp với nhau. Trọng số của mô hình thay đổi trong quá trình này.
- **Modern RAG (Hiện nay):** Là hướng tiếp cận dựa trên In-Context Learning. Với sự bùng nổ của các LLM cực lớn có khả năng hiểu ngữ cảnh rộng, RAG hiện đại thường ám chỉ quy trình “Retrieve and Prompt”.

Trong mô hình hiện đại, chúng ta thường giữ nguyên trọng số của LLM và chỉ tập trung tối ưu hóa việc truy xuất dữ liệu, sau đó đưa dữ liệu này vào đầu vào (Prompt) để mô hình xử lý. Cách tiếp cận này linh hoạt, chi phí thấp và dễ dàng áp dụng cho dữ liệu riêng tư mà không cần quy trình huấn luyện phức tạp.

II.2. Kiến trúc RAG hiện đại

Một hệ thống RAG tiêu chuẩn hiện nay thường được mô hình hóa thành một quy trình gồm 3 giai đoạn chính: Indexing, Retrieval và Generation.



Hình 3: Sơ đồ luồng hoạt động cơ bản của RAG.

II.2.1. Phase 1: Indexing

Giai đoạn này tương đồng với quy trình ETL (Extract-Transform-Load) trong kỹ thuật dữ liệu. Mục tiêu của giai đoạn này là chuyển đổi dữ liệu thô từ nhiều định dạng khác nhau thành một định dạng thống nhất để hệ thống có thể tìm kiếm được.

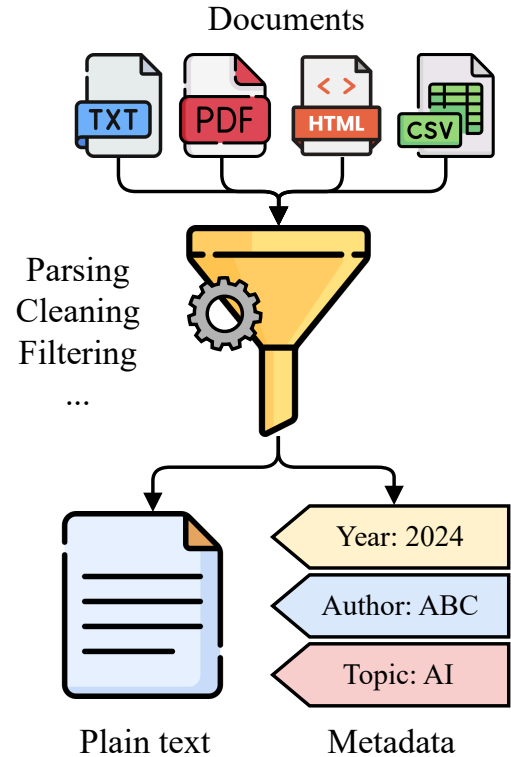
Quy trình này bao gồm các bước từ cơ bản đến nâng cao như sau:

1. Document Loading:

Bắt đầu với việc thu thập nguồn dữ liệu đầu vào, có thể là dữ liệu đã có sẵn trong nội bộ doanh nghiệp hoặc dữ liệu được cào từ internet.

- **Trích xuất nội dung:** Hệ thống cần có khả năng xử lý đa dạng các loại tệp tin để có thể loại bỏ các định dạng hiển thị phức tạp như font chữ, màu sắc, layout và giữ lại phần quan trọng nhất là văn bản thuần túy.
- **Thu thập siêu dữ liệu:** Trong các ứng dụng thực tế, chỉ lấy nội dung văn bản là chưa đủ. Hệ thống RAG tối ưu cần trích xuất kèm cả các thông tin ngữ cảnh đi kèm tài liệu, hay còn gọi là metadata, ví dụ: chủ đề, số trang, ngày xuất bản, tác giả, ...

Vai trò của metadata là cực kỳ quan trọng để hỗ trợ tính năng Pre-filtering. Ví dụ như nếu người dùng hỏi về “Doanh thu năm 2024”, hệ thống sẽ dùng metadata để lọc đúng các tài liệu trong năm 2024 thay vì tìm kiếm toàn bộ dữ liệu.

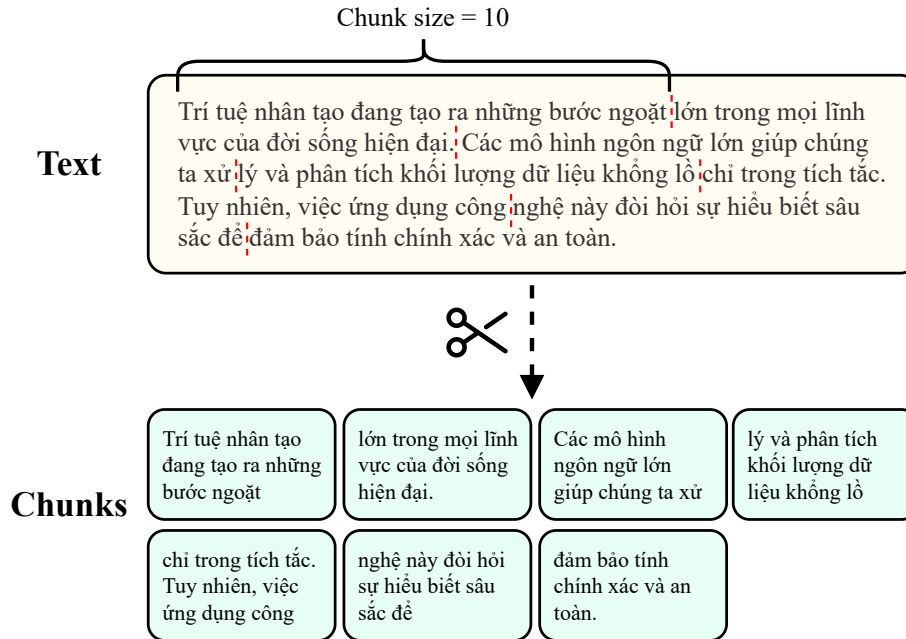


Hình 4: Minh họa quy trình rút trích văn bản từ các nguồn dữ liệu.

2. Text Splitting (Chunking): Đây là bước chia nhỏ tài liệu dài thành các đoạn nhỏ hơn gọi là “chunks”.

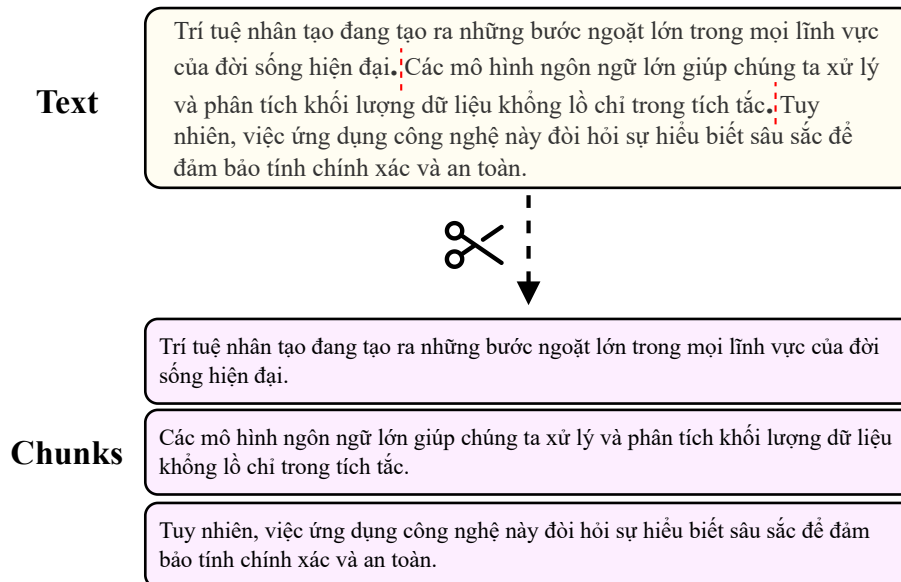
• Tại sao cần Chunking?

- Giới hạn Context Window:** Các mô hình LLM và Embedding đều có giới hạn về số lượng token đầu vào. Không thể đưa toàn bộ nội dung vào mô hình cùng lúc.
 - Độ chính xác của tìm kiếm:** Vector của một đoạn văn ngắn tập trung vào một ý tưởng cụ thể sẽ đại diện cho ngữ nghĩa tốt hơn so với vector trung bình cộng của cả một trang giấy chứa nhiều chủ đề hỗn tạp.
- **Fixed-size Chunking:** Chia văn bản dựa trên số lượng ký tự hoặc token cố định (ví dụ: cứ 500 ký tự thì cắt một lần). Cách này đơn giản nhưng dễ làm mất ngữ nghĩa nếu điểm cắt rơi vào giữa câu hoặc giữa một ý đang trình bày dở.



Hình 5: Minh họa Fixed-size Chunking với Chunk size = 10.

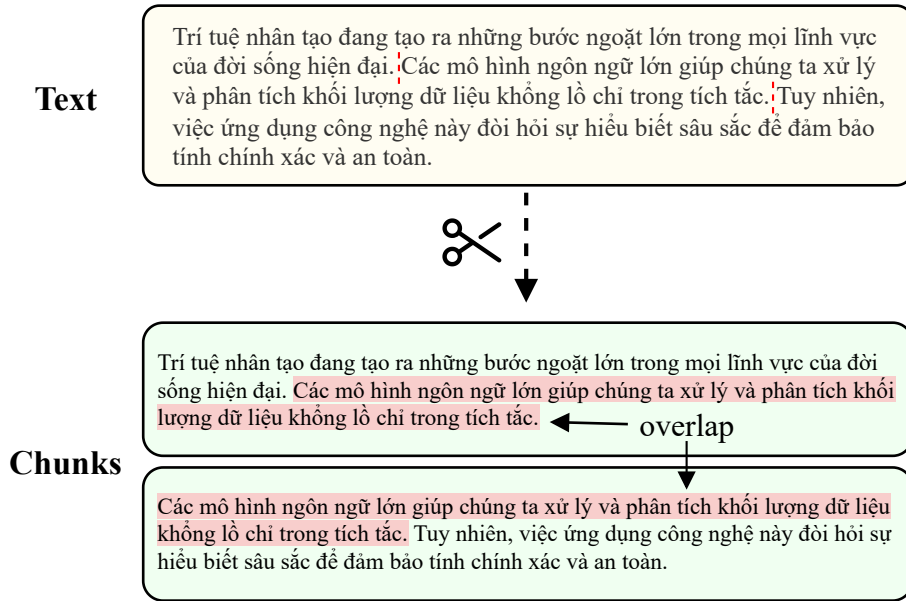
- **Recursive Chunking:** Đây là phương pháp phổ biến nhất hiện nay. Hệ thống sẽ cố gắng cắt dựa trên cấu trúc tự nhiên của văn bản theo thứ tự ưu tiên: dấu ngắt đoạn văn ($\backslash n \backslash n$) $\rightarrow$ dấu xuống dòng ($\backslash n$) $\rightarrow$ dấu chấm câu $\rightarrow$ khoảng trắng. Cách này giúp giữ trọn vẹn cấu trúc câu và đoạn văn.



Hình 6: Minh họa Recursive Chunking bằng dấu chấm câu.

- **Chunk Overlap:** Để đảm bảo ngữ nghĩa không bị mất tại điểm cắt giữa hai chunk liên kề, ta thiết lập tham số `chunk_overlap` (thường là 10-20% độ dài của chunk).

Ví dụ: Chunk 1 kết thúc ở từ thứ 100, thì Chunk 2 sẽ bắt đầu từ từ thứ 80. Phần giao thoa này đóng vai trò như “cầu nối” ngữ cảnh.



Hình 7: Minh họa cho khái niệm Chunk Overlap.

3. Chiến lược Indexing nâng cao

Ở RAG cơ bản, đoạn văn bản dùng để tìm kiếm và đoạn văn bản đưa cho LLM cùng là một. Tuy nhiên, điều này có thể nảy sinh mâu thuẫn: chunk nhỏ tốt cho tìm kiếm, chunk lớn tốt cho LLM do có nhiều ngữ cảnh hơn. Để giải quyết, ta có các chiến lược mở rộng:

- **Parent-Child Indexing (Small-to-Big):**
 - Chia văn bản thành các khối lớn, ví dụ 1000 tokens, gọi là Parent chunks để chứa ngữ cảnh đầy đủ.
 - Chia Parent chunks thành các khối nhỏ hơn, ví dụ 200 tokens, gọi là Child chunks.
 - **Cơ chế:** Hệ thống sẽ index và tìm kiếm trên Child chunks để đạt độ chính xác cao, nhưng khi trả về kết quả cho LLM, hệ thống sẽ lấy Parent chunk tương ứng.
- **Summary Indexing:** Sử dụng LLM để tóm tắt chunk gốc. Ta sẽ index phiên bản tóm tắt này với ý nghĩa được cô đọng hơn, nhưng khi cần sẽ trả về văn bản gốc chi tiết.

4. Embedding

- Sử dụng một mô hình Embedding để chuyển đổi các chunks văn bản thành các vector số học dạng dense vector trong không gian nhiều chiều.
- Theo đó, các đoạn văn bản có nội dung ngữ nghĩa giống nhau sẽ có vector nằm gần nhau trong không gian này.

5. Vector Store

- Lưu trữ các vector cùng với ID và metadata tương ứng vào cơ sở dữ liệu vector chuyên dụng để phục vụ việc truy xuất ở giai đoạn sau.

II.2.2. Phase 2: Retrieval

Đây là thành phần quyết định sự thành bại của hệ thống RAG. Nếu bước này trích xuất thông tin sai hoặc thiếu, LLM sẽ không có đủ dữ liệu để trả lời. Quá trình này không chỉ đơn thuần là tìm kiếm vector mà còn bao gồm các kỹ thuật tối ưu hóa sau:

1. Query Processing

- Ở bước này, hệ thống nhận câu hỏi từ người dùng và đưa qua mô hình Embedding để tạo ra vector truy vấn q .
- Trong thực tế, câu hỏi của người dùng thường ngắn, thiếu ngữ cảnh hoặc có thể mang nhiều nghĩa. Để khắc phục, ta có thể áp dụng:
 - Multi-Query: Sử dụng LLM để sinh ra 3-5 biến thể khác nhau của câu hỏi gốc, sau đó tìm kiếm tất cả và gộp kết quả lại. Điều này giúp tăng khả năng tìm thấy tài liệu liên quan mà không phụ thuộc vào cách diễn đạt duy nhất của người dùng.



Ví dụ:

Input (User): “Lỗi kết nối db” (*Câu hỏi ngắn, thiếu ngữ cảnh cụ thể*)

LLM Generated Queries:

- “Cách khắc phục lỗi connection timeout khi kết nối database.”
- “Xử lý lỗi Access Denied cho user root trong MySQL/PostgreSQL.”
- “Hướng dẫn kiểm tra firewall chặn port 5432 hoặc 3306.”

→ Hệ thống sẽ tìm kiếm cả 3 vấn đề (*Timeout, Permission, Network*) để đảm bảo không bỏ sót tài liệu kỹ thuật liên quan.

- HyDE (Hypothetical Document Embeddings): Yêu cầu LLM viết một câu trả lời giả định cho câu hỏi, sau đó dùng vector embedding của câu trả lời giả định này để tìm kiếm. Cách này giúp thu hẹp khoảng cách ngữ nghĩa giữa “câu hỏi” và “tài liệu chứa câu trả lời”.



Ví dụ:

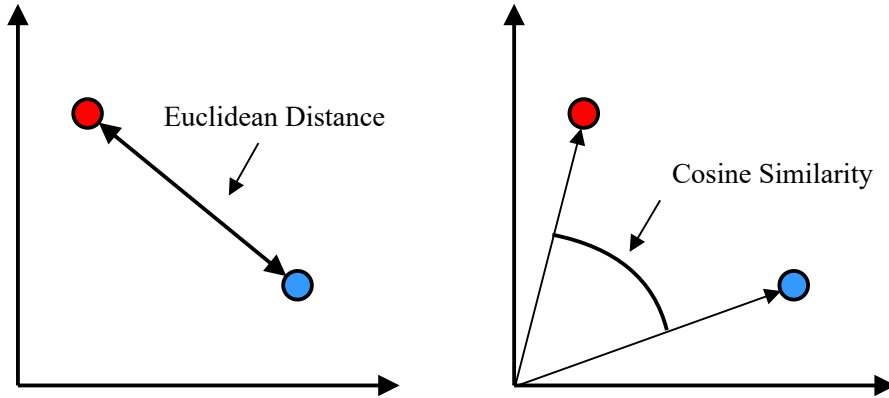
Input (User): “Quy định mang laptop ra ngoài”

Hypothetical Document (LLM giả định): “Theo chính sách bảo mật thông tin, nhân viên muốn mang thiết bị tài sản công ty (laptop, tablet) ra khỏi văn phòng cần điền phiếu ‘Đăng ký mang thiết bị’ trên hệ thống và phải được Trưởng bộ phận phê duyệt...”

→ Vector của đoạn văn giả định này sẽ khớp tốt với tài liệu “Sổ tay nhân viên” hoặc “Quy trình an ninh” trong database, tốt hơn nhiều so với câu hỏi gốc.

2. Similarity Search

- Dense Retrieval: Hệ thống tính toán độ tương đồng giữa vector truy vấn q và các vector tài liệu d trong cơ sở dữ liệu.



Hình 8: Minh họa hai metric đo độ tương đồng hoặc khoảng cách thường được dùng.

- Metric: Sử dụng Cosine Similarity hoặc Euclidean Distance.
- Algorithm: Để đảm bảo tốc độ khi dữ liệu lớn, thay vì so khớp từng đôi một (thường được gọi là Brute-force), ta sử dụng thuật toán ANN (Approximate Nearest Neighbor) như HNSW để tìm kiếm lân cận gần đúng với độ trễ thấp.
- **Vấn đề:** Tìm kiếm vector rất tốt về ngữ nghĩa nhưng lại yếu về từ khóa chính xác. Ví dụ: vector của “iPhone 14” và “iPhone 15” rất gần nhau về ngữ nghĩa, nhưng người dùng cần tìm chính xác thông tin về phiên bản 15.

3. Hybrid Search

Đây là kỹ thuật mở rộng tiêu chuẩn để khắc phục nhược điểm trên.

- **Cơ chế:** Kết hợp kết quả từ hai luồng tìm kiếm song song:
 - Dense Retrieval (Vector Search): Tìm kiếm dựa trên độ tương đồng ngữ nghĩa trong không gian vector. Phương pháp này giải việc tìm kiếm các nội dung có ý nghĩa tương đương (ví dụ: “giá xe” ↔ “chi phí mua ô tô”).
 - Sparse Retrieval (Keyword Search): Sử dụng các thuật toán thống kê truyền thống như BM25 hoặc TF-IDF.
 - *Giải thích:* Các thuật toán này hoạt động bằng cách đếm tần suất xuất hiện chính xác của từ khóa. Nó cực kỳ quan trọng để bắt chính xác các tên riêng, từ khóa hoặc thuật ngữ chuyên ngành mà Vector Search thường bỏ qua.
- **Reciprocal Rank Fusion (RRF):** Là thuật toán hậu xử lý dùng để gộp và xếp hạng lại kết quả từ hai luồng trên.

i Cách hoạt động của thuật toán RRF

Vấn đề: Điểm số của Vector Search (thường là 0.7 - 0.9) và BM25 (thường là 10 - 50) có thang đo hoàn toàn khác nhau, việc cộng trực tiếp sẽ không hợp lí.

Giải pháp: RRF bỏ qua điểm số gốc và chuẩn hóa dựa trên **thứ hạng**.

$$Score = \sum_i \frac{1}{k + r_i}$$

- Cơ chế: Sử dụng nghịch đảo của thứ hạng (r_i). Hạng 1 (mẫu số nhỏ) sẽ cho điểm rất lớn, Hạng 100 (mẫu số lớn) sẽ cho điểm rất nhỏ.
- Hằng số k : Thường là 60, giúp làm mượt điểm số, ngăn không cho các tài liệu Top 1 chiếm ưu thế quá tuyệt đối so với Top 2, Top 3.
- Kết quả: Tài liệu đứng hạng trung bình ở cả 2 danh sách thường có tổng điểm cao hơn tài liệu chỉ đứng nhất ở 1 danh sách nhưng mất tích ở danh sách kia.

4. Re-ranking

Sau khi có được tập ứng viên (ví dụ: Top 50 tài liệu) từ bước trên, thứ tự của chúng có thể chưa hoàn toàn chính xác do vector chỉ là dạng nén thông tin.

- **Cross-Encoder:** Sử dụng một mô hình Deep Learning chuyên biệt để chấm điểm lại mức độ liên quan giữa câu hỏi và từng tài liệu trong tập ứng viên ban đầu.
- **Tại sao cần Cross-Encoder?**
 - Bi-Encoder (Dùng ở bước Indexing/Retrieval): Mã hóa câu hỏi và tài liệu thành 2 vector riêng biệt một cách độc lập. Ưu điểm là cực nhanh, nhưng nhược điểm là làm mất mối quan hệ ngữ pháp và ngữ nghĩa phức tạp giữa câu hỏi và văn bản.
 - Cross-Encoder (Dùng ở bước Re-ranking này): Đưa cả câu hỏi và văn bản vào mô hình cùng một lúc (như con người đọc song song). Nó có thể nhận biết các sắc thái phủ định, quan hệ nguyên nhân-kết quả phức tạp. Chính xác hơn nhưng chậm hơn nhiều.
- **Chiến lược “Hình phễu”:** Retrieve Many (Lấy 50 tài liệu nhanh bằng Bi-Encoder) → Re-rank Few (Lấy 5 tài liệu tốt nhất bằng Cross-Encoder) → Đưa vào LLM. Cách này cân bằng giữa tốc độ và độ chính xác.

💡 Ví dụ:

Query (User): “Tại sao tôi không nhận được thông báo qua email?”

1. Kết quả từ Vector Search (Bi-Encoder): (Tìm dựa trên độ tương đồng chung về chủ đề “email” và “thông báo”)

- Rank 1: “Hướng dẫn cài đặt chữ ký email.” (Sai: cùng chủ đề nhưng sai ý định)
- Rank 2: “Quy định về văn hóa gửi email công ty.” (Sai)
- Rank 3: “Xử lý lỗi email bị rơi vào thư mục Spam.” (Đúng, nhưng bị xếp hạng thấp)

2. Sau khi qua Cross-Encoder (Re-ranking): (Mô hình đọc hiểu mối quan hệ “không nhận được” → “lỗi/spam”)

- A. Rank 1: “Xử lý lỗi email bị rơi vào thư mục Spam.” ↑ (Được đẩy lên đầu)
- B. Rank 2: “Hướng dẫn cài đặt chữ ký email.”
- C. Rank 3: “Quy định về văn hóa gửi email công ty.”

II.2.3. Phase 3: Generation

Sau khi Retrieval trả về danh sách các tài liệu liên quan, hệ thống chuyển sang giai đoạn sinh văn bản. Đây là lúc LLM tổng hợp thông tin để trả lời người dùng. Quá trình này không chỉ đơn giản là đưa dữ liệu vào, mà còn đòi hỏi các chiến lược xử lý ngữ cảnh tinh tế.

1. Context Preparation

- Context Stuffing: Đây là phương pháp đơn giản nhất. Hệ thống gộp toàn bộ văn bản của các tài liệu tìm được (Top-K) thành một đoạn văn dài, sau đó ghép vào Prompt.
- Vấn đề:
 - Chi phí & Độ trễ: Input càng dài, LLM xử lý càng chậm và tốn token.
 - Nhiều thông tin: Quá nhiều thông tin không liên quan có thể khiến LLM bị “loãng” chủ đề.
- Context Selection & Compression:
 - Context Reordering: Dựa trên hiện tượng “Lost in the Middle”: LLM thường chú ý tốt nhất vào thông tin ở đầu và cuối prompt, nhưng hay bỏ quên thông tin ở giữa.

i Chiến thuật Reordering: Hình chữ U

Thay vì đưa tài liệu vào theo thứ tự độ liên quan giảm dần ($1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$), ta sắp xếp lại để các tài liệu quan trọng nhất nằm ở hai đầu mút của Prompt:

Input (Xếp hạng bởi Vector Search):

$[Doc_1] \quad [Doc_2] \quad [Doc_3] \quad [Doc_4] \quad [Doc_5]$

↓

Sắp xếp lại để đưa vào LLM Prompt:

$[Doc_1] \quad [Doc_3] \quad [Doc_5] \quad [Doc_4] \quad [Doc_2]$

→ Kết quả: Doc 1 (quan trọng nhất) nằm đầu, Doc 2 (quan trọng nhì) nằm cuối. Các Doc ít quan trọng hơn (3, 4, 5) được đưa vào giữa để tận dụng tối đa sự tập trung của mô hình.

- Context Compression: Sử dụng một LLM nhỏ hoặc thuật toán NLP để tóm tắt ý chính trước khi đưa vào LLM chính.

**Ví dụ:**

Query: “Phí hủy dịch vụ là bao nhiêu?”

1. Raw Context (300 tokens): “...Theo điều khoản 7.2 của hợp đồng dịch vụ được ký kết vào ngày... (văn bản pháp lý dài dòng)... trong trường hợp khách hàng muốn chấm dứt hợp đồng trước thời hạn, cần thông báo bằng văn bản trước 30 ngày và chịu khoản phí phạt tương đương 02 tháng cước sử dụng...”

2. Compressed Context (20 tokens): “Điều khoản 7.2: Phí hủy dịch vụ là 02 tháng cước sử dụng.”

→ LLM chính chỉ cần nhận đoạn ngắn gọn này, giúp trả lời nhanh hơn và giảm thiểu hallucination.

2. Prompt Engineering

Chất lượng câu trả lời phụ thuộc rất lớn vào cách ta ra lệnh cho mô hình. Một context tốt nhưng prompt tệ vẫn có thể dẫn đến câu trả lời sai.

- **Zero-shot Prompting:** Sử dụng một template cố định để hướng dẫn mô hình trả lời trực tiếp dựa trên ngữ cảnh mà không cần ví dụ mẫu.

RAG Prompt Template (Standard)

System: You are an assistant for question-answering tasks. Use the following pieces of context to answer the question. If you don't know the answer, just say that you don't know.

Context: {context}

Question: {question}

Answer:

- **Few-shot Learning:** Cung cấp thêm 1-2 ví dụ mẫu (gồm bộ ba Context - Question - Answer chuẩn) vào prompt để LLM học được phong cách trả lời và định dạng mong muốn.

**Ví dụ**

Instruction: Trả lời ngắn gọn, chỉ đưa ra con số.

Example 1:

Context: “Doanh thu năm 2022 là 10 tỷ, năm 2023 tăng lên 12 tỷ.”

Question: Doanh thu 2023 bao nhiêu?

Answer: 12 tỷ.

Actual Task:

Context: {real_context}

Question: {user_question}

Answer:

- **Chain-of-Thought (CoT - Chuỗi suy luận):** Yêu cầu mô hình suy nghĩ từng bước (*Let's think step by step*) dựa trên các dữ kiện trong context trước khi đưa ra kết luận cuối cùng.

Tại sao cần CoT?

Đối với các câu hỏi phức tạp đòi hỏi logic, nếu ép LLM trả lời ngay lập tức, nó rất dễ đoán mò.

Với CoT: LLM tự sinh ra quy trình: “*Bước 1: Tìm giá gói A... Bước 2: Tìm giá gói B... Bước 3: So sánh...*” → Kết luận chính xác hơn đáng kể^a.

^aXem chi tiết tại: <https://arxiv.org/abs/2201.11903>

3. Generation & Attribution

- **Cơ bản:** LLM sinh ra câu trả lời dưới dạng văn bản thông thường. Mục tiêu chính là tính trôi chảy và đúng ngữ pháp.
- **Citation:** Một trong những lợi thế cạnh tranh lớn nhất của RAG so với các chatbot truyền thống là khả năng minh bạch hóa nguồn tin.
 - Cơ chế: Trong prompt, ta yêu cầu LLM: “Mọi thông tin đưa ra phải kèm theo ID của tài liệu nguồn”.
 - Lợi ích: Giúp người dùng dễ dàng kiểm chứng, xây dựng niềm tin và giảm thiểu rủi ro khi mô hình bịa đặt thông tin.

Minh họa output có trích dẫn nguồn

User Question: “Quy định làm thêm giờ tính thế nào?”

RAG Response: “Theo quy định công ty, nhân viên làm thêm giờ vào ngày thường được hưởng 150% lương cơ bản [**Sổ tay nhân viên, Tr.12**]. Đối với ngày lễ tết, mức hưởng là 300% lương [**Luật lao động 2019, Điều 98**].”

→ Người dùng có thể click vào [...] để mở trực tiếp tài liệu gốc đối chiếu.

III. Thư viện LangChain

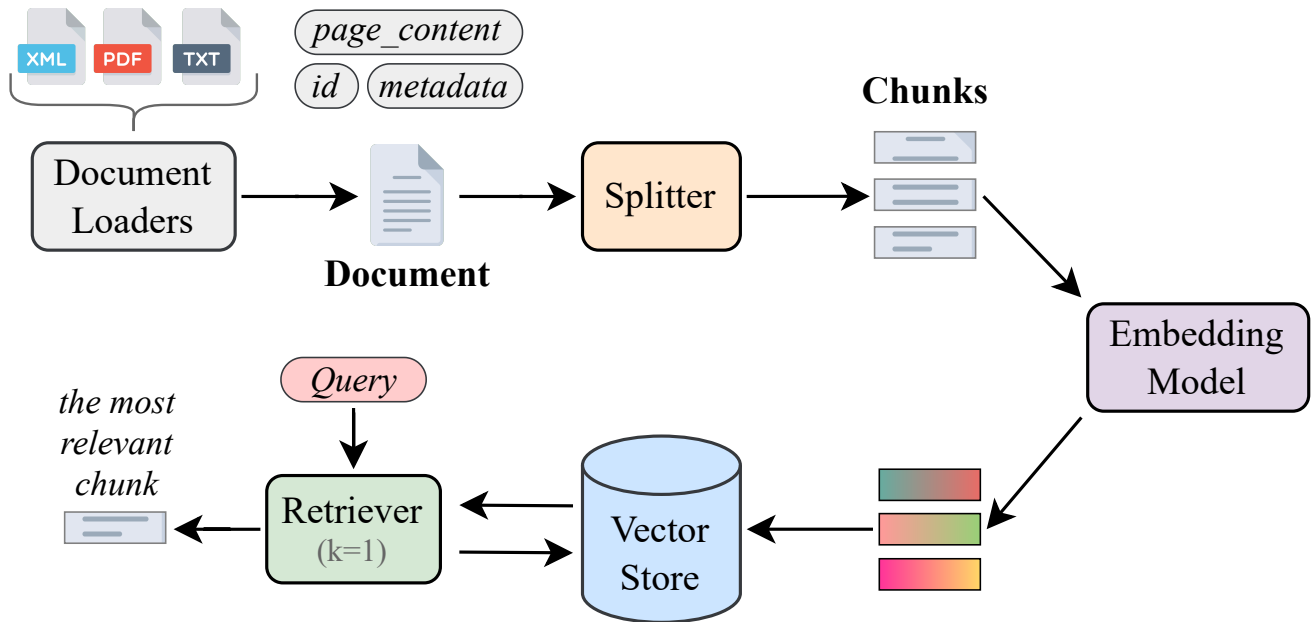
III.1. Giới thiệu

LangChain là một framework mã nguồn mở mạnh mẽ, được thiết kế để đơn giản hóa quá trình phát triển các ứng dụng sử dụng mô hình ngôn ngữ lớn (LLM). Một cách tổng quát, LangChain đóng vai trò liên kết các thành phần dữ liệu, xử lý logic và mô hình lại với nhau.

Trong bài toán Retrieval Augmented Generation (RAG), LangChain giải quyết vấn đề cốt lõi là sự rời rạc giữa nguồn dữ liệu và khả năng của LLM. Thư viện này hỗ trợ toàn bộ vòng đời của ứng dụng RAG mà không cần người lập trình phải tự xây dựng các kết nối thủ công phức tạp.

III.2. Các thành phần cốt lõi

Để xây dựng một ứng dụng RAG với LangChain, ta cần phải hiểu về các thành phần chính như bên dưới:



Hình 9: Tổng quan các thành phần chính của RAG trong LangChain.

1. Documents và Document Loaders

Trong LangChain, đơn vị cơ bản để biểu diễn thông tin là đối tượng **Document**. Trước khi xây dựng RAG, dữ liệu từ các nguồn khác nhau cần được chuyển đổi về định dạng này.

Cấu trúc Document: LangChain triển khai lớp **Document** để đại diện cho một đơn vị văn bản và các siêu dữ liệu đi kèm, thường được gọi là **metadata**. Một đối tượng **Document** bao gồm ba thuộc tính chính:

- **page_content:** Chuỗi ký tự chứa nội dung văn bản.

- **metadata:** Một dictionary chứa thông tin bổ sung tùy ý, ví dụ như nguồn gốc của tài liệu, số trang, quan hệ với tài liệu khác.
- **id:** Mã định danh chuỗi cho tài liệu.

Loading Documents: LangChain cung cấp hệ sinh thái **Document Loaders** tích hợp với hàng trăm nguồn dữ liệu khác nhau như PDF, CSV, HTML. Ví dụ, sử dụng **PyPDFLoader** để nạp một tài liệu PDF thành một **Document**.

```
1 from langchain_community.document_loaders import PyPDFLoader
2
3 file_path = "../ai-vietnam-2025.pdf"
4 loader = PyPDFLoader(file_path)
5 doc = loader.load()
```

Splitting: Một trang tài liệu gốc thường quá dài hoặc chứa nhiều thông tin hỗn tạp. Việc chia nhỏ văn bản giúp tăng độ chính xác khi truy xuất. Công cụ **Text Splitters** được sử dụng để chia tài liệu thành các đoạn nhỏ, các đoạn này được gọi là các “chunk”. Phương pháp phổ biến là **RecursiveCharacterTextSplitter**, với chức năng:

- Phân chia đệ quy dựa trên các ký tự phân cách như xuống dòng, khoảng trắng, để giữ nguyên vẹn ngữ nghĩa.
- Giữ lại một phần ký tự trùng lặp giữa các đoạn liên kế để đảm bảo ngữ cảnh không bị ngắt quãng.

```
1 from langchain_text_splitters import RecursiveCharacterTextSplitter
2
3 text_splitter = RecursiveCharacterTextSplitter(
4     chunk_size=1000, # Kích thước của mỗi chunk
5     chunk_overlap=200, # Giữ 200 tokens của đoạn liền kề
6 )
7 all_splits = text_splitter.split_documents(doc)
```

2. Embeddings

Vector Search là phương pháp phổ biến để lưu trữ và tìm kiếm trên dữ liệu phi cấu trúc.

Nguyên lý hoạt động: Mô hình Embedding chuyển đổi một văn bản thành một vector số thực biểu diễn cho văn bản ấy. Việc biểu diễn dưới dạng số học cho phép:

- Những văn bản có ý nghĩa tương tự sẽ có vector gần nhau trong không gian hình học.
- Các chỉ số đo lường như cosine similarity được dùng để xác định độ tương đồng giữa các văn bản thông qua tính toán trên vector tương ứng của chúng.

LangChain hỗ trợ giao diện chuẩn cho nhiều nhà cung cấp mô hình embedding như OpenAI, Google, HuggingFace, ...


```

1 from langchain_openai import OpenAIEmbeddings
2
3 embeddings = OpenAIEmbeddings(model="text-embedding-3-large")
4
5 # Tạo vector cho một đoạn văn bản
6 vector_1 = embeddings.embed_query(all_splits[0].page_content)
7 print(len(vector_1))
8 # Kết quả: 1536 (số chiều của vector)

```

3. Vector Stores

VectorStore trong LangChain chịu trách nhiệm lưu trữ các đối tượng **Document** và vector tương ứng, đồng thời cung cấp phương thức để truy vấn tới chúng. Gồm hai chức năng chính:

- **Indexing:** Thêm văn bản vào kho lưu trữ thông qua phương thức `add_documents`.
- **Querying:** Tìm kiếm văn bản dựa trên độ tương đồng giữa hai vector với phương thức `similarity_search`.

LangChain tích hợp với nhiều loại vector store: từ dạng lưu trong bộ nhớ như **FAISS** và **InMemoryVectorStore**, đến các cơ sở dữ liệu chuyên dụng với **Chroma**, **Pinecone**, **Postgres**.

```

1 from langchain_core.vectorstores import InMemoryVectorStore
2
3 vector_store = InMemoryVectorStore(embeddings)
4 ids = vector_store.add_documents(documents=all_splits)
5 results = vector_store.similarity_search(
6     "How many distribution centers does Nike have in the US?"
7 )

```

4. Retrievers

Trong khi **VectorStore** là nơi lưu trữ, **Retriever** được dùng để truy vấn dữ liệu. Điểm đặc biệt là Retrievers trong LangChain là các **Runnable**, cho phép chúng kết nối dễ dàng vào các chuỗi xử lý hay “chains”.

VectorStoreRetriever

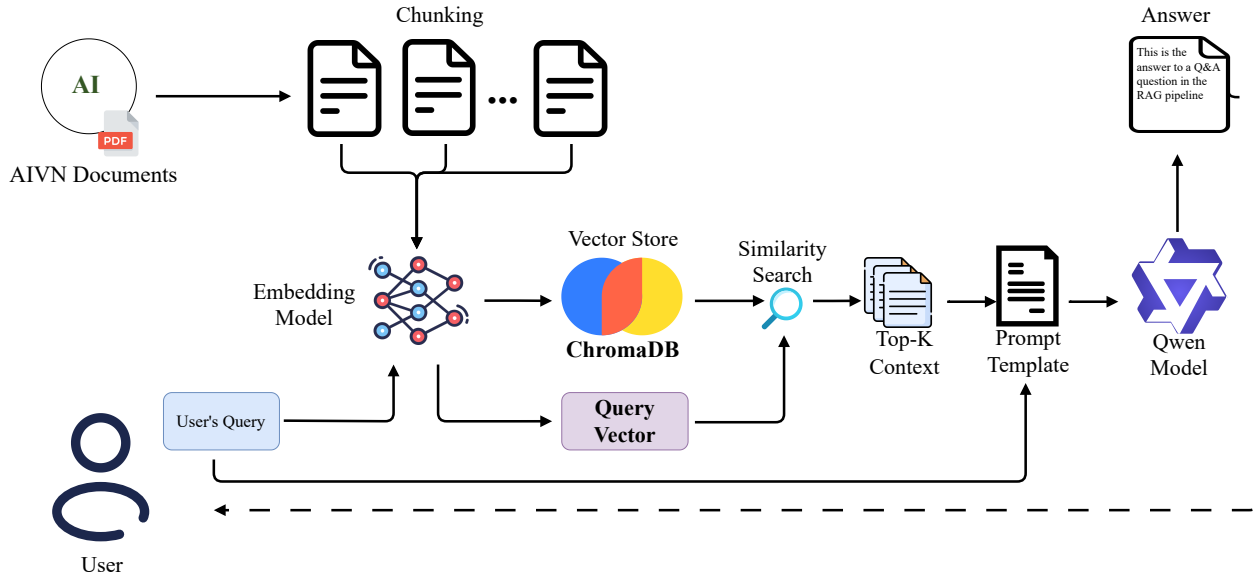
Cách để tạo Retriever đơn giản nhất là từ một **VectorStore** thông qua hàm `.as_retriever()`. Trong đó, có thể bổ sung các tham số tìm kiếm như `search_kwargs` và loại tìm kiếm `search_type`:

- `similarity`: Tìm kiếm tương đồng mặc định.
- `mmr` (Maximum Marginal Relevance): Cân bằng giữa độ tương đồng và độ đa dạng của kết quả.
- `similarity_score_threshold`: Lọc bỏ các kết quả có điểm tương đồng thấp hơn ngưỡng quy định.

```
1 # Tạo Retriever từ Vector Store
2 retriever = vector_store.as_retriever(
3     search_type="similarity",
4     search_kwargs={"k": 1} # Chỉ lấy 1 kết quả tốt nhất
5 )
6
7 # Thực thi tìm kiếm (Batch)
8 retriever.batch([
9     "How many distribution centers does Nike have in the US?",
10    "When was Nike incorporated?"
11 ])
```

IV. Thực hành

Trong phần này, chúng ta sẽ hiện thực hoá các khái niệm về RAG và LangChain bằng việc xây dựng một Question Answering System (tạm dịch: hệ thống hỏi đáp) dành cho các tài liệu PDF tiếng Việt.



Hình 10: Kiến trúc hệ thống RAG sử dụng ChromaDB và mô hình ngôn ngữ Qwen.

Quan sát Hình 10, quy trình phần thực hành bắt đầu từ việc xử lý tài liệu đầu vào, sau đó thực hiện chia nhỏ và vector hoá để lưu trữ vào ChromaDB. Khi người dùng đặt câu hỏi, hệ thống sẽ tìm kiếm ngữ cảnh phù hợp và đưa vào mô hình Qwen để sinh câu trả lời.

Về mặt kỹ thuật, chúng ta sẽ làm việc trên Google Colab với GPU T4, Python phiên bản 3.12, sử dụng nguồn dữ liệu từ các bài viết public của AI VIET NAM. Quy trình triển khai chi tiết sẽ đi qua các bước: thiết lập môi trường làm việc và tổ chức cấu trúc dự án, thu thập và xử lý dữ liệu, xây dựng cơ sở dữ liệu vector, khởi tạo mô hình ngôn ngữ và cuối cùng là phát triển giao diện tương tác.

Các giai đoạn dưới đây được trình bày theo đúng thứ tự của các cell trong notebook, giúp bạn vừa nắm được ý tưởng, vừa dễ dàng chạy thử từng đoạn mã minh họa.

IV.1. Chuẩn bị môi trường và cấu trúc dự án

Ở bước đầu tiên, chúng ta thiết lập môi trường làm việc và tổ chức lại cấu trúc thư mục nhằm đảm bảo toàn bộ mã nguồn vận hành có hệ thống, dễ mở rộng và dễ tích hợp các thành phần của pipeline RAG trong các bước tiếp theo.

```

1 # Cài đặt các thư viện chính cho mô hình ngôn ngữ, embedding, RAG và giao diện web
2 !pip install -q \
3     "torch>=2.0.0" \
4     "transformers>=4.40.0" \
5     "accelerate>=0.30.0" \
6     "huggingface-hub>=0.23.0" \
7     "sentence-transformers>=2.7.0" \
8     "langchain>=0.2.0" \
9     "langchain-core>=0.2.0" \
10    "langchain-community>=0.1.0" \
11    "langchain-text-splitters>=0.2.0" \
12    "chromadb>=0.5.0" \
13    "langchain-chroma>=0.2.0" \
14    "pypdf>=4.2.0" \
15    "langserve[all]>=0.1.0" \
16    "fastapi>=0.115.0" \
17    "uvicorn>=0.30.0" \
18    "gradio>=5.0.0" \
19    "langchain-huggingface" \
20    "wget"

```

Đoạn mã sau giúp thiết lập thư mục gốc của dự án trên Colab và tạo các thư mục con để lưu dữ liệu và mã nguồn. Cách tổ chức này giúp chúng ta dễ quản lý khi số lượng file tăng lên.

```

1 import os
2 import sys
3
4 PROJECT_ROOT = "/content/rag_langchain"
5
6 # Đặt token để tải mô hình private, hãy thay bằng token của bạn
7 os.environ["HF_TOKEN"] = "YOUR_HUGGINGFACE_TOKEN"
8 # Tạo thư mục cho dữ liệu và mã nguồn
9 os.makedirs(os.path.join(PROJECT_ROOT, "data_source", "generative_ai"), exist_ok=True)
10 os.makedirs(os.path.join(PROJECT_ROOT, "src", "base"), exist_ok=True)
11 os.makedirs(os.path.join(PROJECT_ROOT, "src", "rag"), exist_ok=True)
12
13 os.chdir(PROJECT_ROOT)
14
15 # Thêm PROJECT_ROOT vào sys.path để có thể import các module trong src
16 if PROJECT_ROOT not in sys.path:
17     sys.path.append(PROJECT_ROOT)
18
19 os.environ["TOKENIZERS_PARALLELISM"] = "false"

```

Cuối cùng, chúng ta tạo các tệp `__init__.py` để các thư mục mã nguồn có thể được sử dụng như các package Python, giúp việc import module trở nên rõ ràng hơn.

```

1 %%bash
2 touch /content/rag_langchain/src/__init__.py

```

```
3 touch /content/rag_langchain/src/base/__init__.py
4 touch /content/rag_langchain/src/rag/__init__.py
```

IV.2. Chuẩn bị dữ liệu PDF đầu vào

Tại bước này, ta tải các tài liệu PDF dùng làm dữ liệu cho hệ thống RAG. Các tài liệu này được trích từ các bài viết công khai của AI VIET NAM, sau đó được lưu vào một thư mục chung để phục vụ cho bước xử lý văn bản và xây dựng vector database.

```
1 import os
2 import wget
3
4 # Thư mục lưu các tài liệu PDF
5 DATA_DIR = "/content/rag_langchain/data_source/generative_ai"
6 os.makedirs(DATA_DIR, exist_ok=True)
7
8 # Danh sách các tài liệu PDF mẫu sẽ dùng làm dữ liệu cho hệ thống RAG
9 pdf_links = [
10     {
11         "title": "Vòng lặp for và ứng dụng",
12         "url": "https://docs.google.com/uc?export=download&id=
13             1_zJnj5qORwzMH6vgftGzkYu4fFew6Pq4"
14     },
15     {
16         "title": "Giám sát hệ thống AI với Grafana và Prometheus",
17         "url": "https://docs.google.com/uc?export=download&id=1gZWLJddiuLd-
18             ZZ8j_Nmfu_r7fISGRW6J"
19     },
20     {
21         "title": "Các Thuộc Đo Đánh Giá Mô Hình Hồi Quy",
22         "url": "https://docs.google.com/uc?export=download&id=1C-
23             f9pNW0mkMxaakDcpliN3isTVqRQtR3"
24     },
25     {
26         "title": "A simple, strong baseline for Long-Term Forecasts",
27         "url": "https://docs.google.com/uc?export=download&id=
28             16KFeWi00NqV3ZJYAgC_20y7bxUNGd9hU"
29     },
30 ]
31
32 # Tải các file PDF về thư mục dữ liệu nếu chưa tồn tại
33 for pdf_info in pdf_links:
34     save_path = os.path.join(DATA_DIR, f"{pdf_info['title']}.pdf")
35     if not os.path.exists(save_path):
36         try:
37             wget.download(pdf_info["url"], out=save_path)
38         except Exception as e:
39             pass
```

IV.3. Tiền xử lý văn bản và Chunking

Ở bước này, ta làm sạch văn bản trích xuất từ các file PDF và chia nội dung thành các đoạn nhỏ phù hợp. Việc tiền xử lý giúp văn bản gọn, ít lỗi, còn việc chia đoạn giúp mô hình embedding và mô hình ngôn ngữ làm việc hiệu quả hơn.

Làm sạch văn bản tiếng Việt

Văn bản trích xuất từ PDF thường chứa nhiều ký tự đặc biệt, khoảng trắng thừa, hoặc các vấn đề về encoding. Hàm `clean_vietnamese_text` dưới đây giúp chuẩn hóa văn bản tiếng Việt, loại bỏ các ký tự không mong muốn và đảm bảo encoding đồng nhất.

```

1 import re
2 import unicodedata
3 from typing import List
4
5 def clean_vietnamese_text(text: str) -> str:
6     # Chuẩn hóa Unicode về dạng NFC cho tiếng Việt
7     text = unicodedata.normalize('NFC', text)
8     text = "".join(
9         char for char in text
10        if not unicodedata.category(char).startswith('C') or char in '\n\t'
11    )
12
13    # Gộp khoảng trắng thừa và dòng trống
14    text = re.sub(r'\s+', ' ', text)
15    text = re.sub(r'\n\s*\n', '\n', text)
16    return text.strip()

```

Tải và chia nhỏ tài liệu PDF

Sau khi đã có hàm làm sạch văn bản, ta xây dựng hai class chính: `SimpleLoader` để tải tài liệu PDF và `TextSplitter` để chia văn bản thành các chunk phù hợp với mô hình.

```

1 import glob
2 from tqdm import tqdm
3 from langchain_community.document_loaders import PyPDFLoader
4 from langchain_text_splitters import RecursiveCharacterTextSplitter
5
6 class SimpleLoader:
7     # Tải một file PDF và gọi hàm làm sạch nội dung văn bản
8     def load_pdf(self, pdf_file: str):
9         docs = PyPDFLoader(pdf_file, extract_images=True).load()
10        for doc in docs:
11            doc.page_content = clean_vietnamese_text(doc.page_content)
12        return docs
13

```

```

14  # Tải tất cả file PDF trong một thư mục
15  def load_dir(self, dir_path: str) -> List:
16      pdf_files = glob.glob(f"{dir_path}/*.pdf")
17      if not pdf_files:
18          raise ValueError(f"No PDF files found in {dir_path}")
19
20      all_docs = []
21      for pdf_file in tqdm(pdf_files, desc="Loading PDFs"):
22          try:
23              all_docs.extend(self.load_pdf(pdf_file))
24          except Exception as e:
25              pass
26      return all_docs
27
28  class TextSplitter:
29
30      # Khởi tạo với chunk_size và chunk_overlap phù hợp cho tiếng Việt
31      def __init__(self,
32                  chunk_size: int = 400,
33                  chunk_overlap: int = 120,
34                  ):
35          self.splitter = RecursiveCharacterTextSplitter(
36              separators=["\n\n", "\n", " ", ""],
37              chunk_size=chunk_size,
38              chunk_overlap=chunk_overlap,
39              length_function=len,
40          )
41
42      # Chia danh sách documents thành các chunk nhỏ hơn
43      def split(self, documents):
44          return self.splitter.split_documents(documents)

```

IV.4. Xây dựng Vector Database

Tại bước này, các chunk đã được chia nhỏ sẽ được chuyển đổi thành vector thông qua mô hình embedding đa ngôn ngữ từ HuggingFace, sau đó lưu trữ vào Chroma. Class **VectorDB** lúc này đóng vai trò trung tâm, quản lý toàn bộ quy trình từ khởi tạo embedding model, index các document chunks, cho đến việc cung cấp retriever phục vụ cho việc tìm kiếm ngữ nghĩa.

```

1  from langchain_chroma import Chroma
2  from langchain_huggingface import HuggingFaceEmbeddings
3
4  class VectorDB:
5      def __init__(
6          self,
7          documents=None,
8          embedding_model: str = "sentence-transformers/paraphrase-multilingual-MiniLM-
                               L12-v2",
9          collection_name: str = "vietnamese_docs",

```

```

10     persist_dir: str = "/content/chroma_data",
11 ):
12     self.persist_dir = persist_dir
13     self.collection_name = collection_name
14
15     self.embedding = HuggingFaceEmbeddings(model_name=embedding_model)
16     self.db = self._build_db(documents)
17
18     # Xây dựng hoặc tải Vector Database
19     def _build_db(self, documents):
20         # Trường hợp tải database đã có từ persist_dir
21         if documents is None or len(documents) == 0:
22             db = Chroma(
23                 collection_name=self.collection_name,
24                 embedding_function=self.embedding,
25                 persist_directory=self.persist_dir,
26             )
27
28             # Trường hợp tạo mới database từ documents
29         else:
30             db = Chroma.from_documents(
31                 documents=documents,
32                 embedding=self.embedding,
33                 collection_name=self.collection_name,
34                 persist_directory=self.persist_dir,
35             )
36         return db
37
38     # Tạo retriever để tìm kiếm tương đồng
39     def get_retriever(self, search_kwargs: dict = None):
40         if search_kwargs is None:
41             search_kwargs = {"k": 4} # Mặc định lấy 4 document gần nhất
42
43         return self.db.as_retriever(
44             search_type="similarity",
45             search_kwargs=search_kwargs,
46         )

```

IV.5. Khởi tạo LLM và xây dựng RAG Chain

Ta khởi tạo mô hình ngôn ngữ để đảm nhiệm phần sinh câu trả lời, đồng thời định nghĩa một RAG chain kết hợp retriever, prompt và LLM thành một quy trình thống nhất để trả lời câu hỏi dựa trên ngữ cảnh đã truy xuất.

Khởi tạo LLM

Hàm `get_hf_llm` khởi tạo một mô hình ngôn ngữ từ HuggingFace và đóng gói thành LangChain pipeline. Ở đây, ta sử dụng Qwen2.5-3B-Instruct, một mô hình instruction-tuned với khả năng xử lý hiệu quả trong các tác vụ hỏi đáp và sinh văn bản.


```

1 import torch
2 from transformers import pipeline, AutoTokenizer, AutoModelForCausalLM
3 from langchain_huggingface import HuggingFacePipeline
4
5 def get_hf_llm(
6     model_name: str = "Qwen/Qwen2.5-3B-Instruct",
7     temperature: float = 0.2, max_new_tokens: int = 450,
8     **kwargs
9 ):
10     model = AutoModelForCausalLM.from_pretrained(
11         model_name,
12         torch_dtype=torch.float16,
13         device_map="auto",
14         low_cpu_mem_usage=True
15     )
16
17     tokenizer = AutoTokenizer.from_pretrained(model_name)
18
19     # Tạo text generation pipeline
20     model_pipeline = pipeline(
21         "text-generation",
22         model=model,
23         tokenizer=tokenizer,
24         temperature=temperature,
25         max_new_tokens=max_new_tokens,
26         pad_token_id=tokenizer.eos_token_id,
27         do_sample=True,
28         top_p=0.75
29     )
30
31     # Đóng gói pipeline thành LangChain LLM
32     llm = HuggingFacePipeline(pipeline=model_pipeline, model_kwargs=kwargs)
33     return llm

```

Định nghĩa RAG Chain và parser

Class `FocusedAnswerParser` giúp làm sạch và rút gọn câu trả lời của LLM, loại bỏ các phần thừa hoặc lặp lại. Trong khi đó, class `OfflineRAG` kết nối các thành phần retriever, prompt template và LLM thành một chain hoàn chỉnh.

```

1 from langchain_core.prompts import PromptTemplate
2 from langchain_core.runnables import RunnablePassthrough
3 from langchain_core.output_parsers import StrOutputParser
4
5 class FocusedAnswerParser(StrOutputParser):
6     def parse(self, text: str) -> str:
7         text = text.strip()
8         if "[TRẢ LỜI]:" in text:
9             answer = text.split("[TRẢ LỜI]:")[-1].strip()
10        else:

```

```

11         answer = text
12
13         answer = re.sub(r'^\s*[\-\*]\s*', '', answer, flags=re.MULTILINE)
14         answer = re.sub(r'\n+', ' ', answer)
15         lines = [line.strip() for line in answer.split('. ') if line.strip() and len(
16                                     line.strip()) > 5]
17         return '. '.join(lines[:5]) + ('.' if lines else '')
18
19 class OfflineRAG:
20     def __init__(self, llm):
21         self.llm = llm
22         self.prompt = PromptTemplate.from_template("""
23     Bạn là trợ lý AI phân tích tài liệu tiếng Việt.
24     [TÀI LIỆU]:
25     {context}
26
27     [CÂU HỎI]:
28     {question}
29
30     Hãy trả lời dựa trên tài liệu. Nếu tài liệu không có thông tin, nói rõ "Không có thông
31     tin".
32     Trả lời đầy đủ thông tin (3-5 câu chi tiết), không thêm bất kỳ chi tiết nào ngoài tài
33     liệu.
34     [TRẢ LỜI]: """)
35
36     self.answer_parser = FocusedAnswerParser()
37
38     # Xây dựng RAG chain từ retriever
39     def get_chain(self, retriever):
40         def format_docs(docs):
41             formatted = []
42             seen = set()
43             for doc in docs:
44                 content = doc.page_content.strip()
45                 if content and len(content) > 40 and content not in seen:
46                     formatted.append(content)
47                     seen.add(content)
48             return "\n\n".join(formatted)
49         rag_chain = (
50             {"context": retriever | format_docs, "question": RunnablePassthrough()}
51             | self.prompt
52             | self.llm
53             | self.answer_parser
54         )
55         return rag_chain

```

Kết nối toàn bộ pipeline

Sau khi đã có đủ các thành phần, ta kết nối chúng lại thành một hàm xử lý duy nhất. Sau đó sử dụng hàm `answer_question` thực hiện toàn bộ quy trình: nhận câu hỏi, truy xuất dữ liệu,

xây dựng prompt và sinh câu trả lời cuối cùng.

```

1 os.chdir("/content/rag_langchain")
2
3 # Khởi tạo LLM
4 llm = get_hf_llm()
5
6 data_dir = "/content/rag_langchain/data_source/generative_ai"
7
8 # Load và xử lý documents
9 loader = SimpleLoader()
10 text_splitter = TextSplitter(chunk_size=400, chunk_overlap=120)
11
12 raw_docs = loader.load_dir(data_dir)
13 split_docs = text_splitter.split(raw_docs)
14
15 # Xây dựng Vector Database và Retriever
16 vdb = VectorDB(documents=split_docs)
17 retriever = vdb.get_retriever(search_kwargs={"k": 4})
18
19 # Tạo RAG Chain
20 rag = OfflineRAG(llm)
21 rag_chain = rag.get_chain(retriever)
22
23 # Hàm xử lý câu hỏi
24 def answer_question(question: str) -> str:
25     try:
26         return rag_chain.invoke(question)
27     except Exception as e:
28         return f"Error: {str(e)}"

```

IV.6. Xây dựng giao diện ứng dụng

Cuối cùng, chúng ta xây dựng một giao diện đơn giản sử dụng Gradio để người dùng có thể tương tác trực tiếp với hệ thống RAG. Giao diện này không chỉ tạo điều kiện thuận lợi cho việc trải nghiệm hệ thống mà còn giúp minh họa trực quan toàn bộ quy trình hoạt động của pipeline RAG, từ việc tiếp nhận truy vấn đến khi trả về kết quả.

```

1 import gradio as gr
2
3 # Xây dựng giao diện với Gradio Blocks
4 with gr.Blocks(title="RAG Vietnamese QA") as demo:
5     gr.Markdown("# RAG - Hỏi Đáp về Tài Liệu")
6
7     # Thiết kế giao diện cho các thành phần hiển thị trên 1 dòng
8     with gr.Row():
9         # Cột bên trái: Input câu hỏi
10         with gr.Column(scale=1):
11             question_input = gr.Textbox(

```

```

12         label="Câu hỏi",
13         placeholder="Ví dụ: Vì sao classification lại không thể chỉ nhìn
14             accuracy để đánh giá?",
15         lines=3,
16     )
17     submit_btn = gr.Button("Gửi", variant="primary")
18
19     # Cột bên phải: Output câu trả lời
20     with gr.Column(scale=2):
21         answer_output = gr.Textbox(
22             label="Câu trả lời",
23             lines=6,
24             interactive=False
25         )
26
27     # Kết nối button với hàm xử lý
28     submit_btn.click(
29         fn=answer_question,
30         inputs=question_input,
31         outputs=answer_output,
32     )
33 demo.launch(share=True)

```

RAG - Hỏi Đáp về Tài Liệu

Câu hỏi

Vì sao classification lại không thể chỉ nhìn accuracy để đánh giá?

Gửi

Câu trả lời

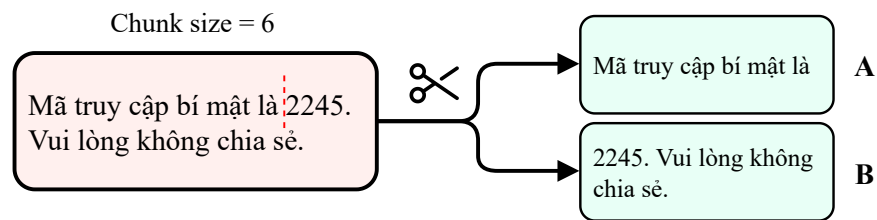
Trong bài toán phân loại, nếu chỉ nhìn vào độ chính xác (accuracy), có thể dẫn đến kết luận sai lầm. Mô hình có thể luôn dự đoán kết quả âm tính cho tất cả các mẫu dữ liệu, dẫn đến độ chính xác đạt 99%, mặc dù mô hình hoàn toàn vô dụng vì không phát hiện được bất kỳ ca bệnh nào. Do đó, chỉ dựa vào độ chính xác không đủ để đánh giá hiệu năng của mô hình, cần phải xem xét thêm các chỉ số khác như độ chính xác cụ thể đối với lớp dương và lớp âm, cũng như độ chính xác cụ thể của mô hình trong việc dự đoán các mẫu có đặc điểm tương tự với các mẫu đã được học. Ngoài ra, độ chính xác không thể phản ánh được độ chính xác của mô hình trong việc loại bỏ các mẫu giả đối hoặc các mẫu ngoại lệ. Vì vậy, trong classification, chỉ nhìn vào độ chính xác là không đủ để đánh giá hiệu năng của mô hình..

Hình 11: Hình ảnh giao diện demo chương trình RAG

Hình 11 minh họa giao diện người dùng sau khi hệ thống được khởi tạo thành công. Khi chạy cell này, một đường link Gradio sẽ được tạo ra, cung cấp môi trường tương tác trực quan. Tại đây, người dùng có thể nhập câu hỏi liên quan đến các tài liệu đã được index, sau đó quan sát cách hệ thống RAG truy xuất ngữ cảnh và sinh ra câu trả lời tương ứng.

V. Câu hỏi trắc nghiệm

- Trong cơ chế Hybrid Search, tại sao chúng ta không thể cộng trực tiếp điểm số từ Vector Search và Keyword Search mà phải cần đến thuật toán Reciprocal Rank Fusion (RRF)?
 - Vì Vector Search trả về kết quả quá chậm so với Keyword Search nên cần RRF để đồng bộ hóa thời gian.
 - Vì thang đo điểm số của hai phương pháp khác nhau, việc cộng trực tiếp sẽ khiến BM25 lấn át hoàn toàn Vector.
 - Vì Vector Search chỉ tìm được từ đồng nghĩa, trong khi RRF giúp loại bỏ các từ đồng nghĩa sai.
 - Vì RRF là thuật toán duy nhất có thể chạy trên GPU để tăng tốc độ xử lý.
- Quan sát hình minh họa dưới đây:



- Nếu người dùng hỏi: “Mã truy cập bí mật là gì?”, kết quả nào sau đây phản ánh chính xác nhất hoạt động của hệ thống RAG?
- Hệ thống tìm thấy Chunk B vì nó chứa con số cụ thể, và LLM trả lời chính xác mã số.
 - Hệ thống tìm thấy Chunk A, nhưng LLM sẽ không trả lời được mã số vì thông tin này nằm ở Chunk B, vốn không được tìm thấy.
 - Hệ thống sẽ tự động tìm thấy cả Chunk A và Chunk B nhờ cơ chế suy luận bắc cầu của Vector Database.
 - Hệ thống không tìm thấy cả hai chunk vì câu văn bị cắt đôi làm hỏng cấu trúc ngữ pháp.
- Tại sao mô hình Cross-Encoder lại có độ chính xác cao hơn Bi-Encoder, mặc dù Cross-Encoder chậm hơn rất nhiều?
 - Vì Cross-Encoder quan sát được sự tương tác trực tiếp giữa từng token của câu hỏi và từng token của tài liệu cùng lúc.
 - Vì Cross-Encoder nén câu hỏi và tài liệu thành các vector có số chiều lớn hơn.
 - Vì Cross-Encoder sử dụng cơ sở dữ liệu đồ thị Graph Database thay vì Vector Database.
 - Vì Cross-Encoder thực hiện việc tìm kiếm từ khóa chính xác thay vì tìm kiếm ngữ nghĩa.
 - Để khắc phục hiện tượng “Lost in the Middle”, chiến lược sắp xếp ngữ cảnh nào sau đây là tối ưu nhất trước khi gửi cho LLM?

- (a) Sắp xếp theo thứ tự độ tương đồng giảm dần.
 - (b) Sắp xếp ngẫu nhiên để tránh việc mô hình bị thiên kiến vào vị trí.
 - (c) Sắp xếp theo hình chữ U: Đưa các tài liệu quan trọng nhất vào đầu và cuối prompt, giấu các tài liệu ít quan trọng hơn vào giữa.
 - (d) Sắp xếp theo trình tự thời gian: Các tài liệu mới nhất được đưa lên đầu, cũ nhất xuống cuối.
5. Trong LangChain, “đơn vị cơ bản” để biểu diễn thông tin là gì?
- (a) Chunk.
 - (b) Vector.
 - (c) PromptTemplate.
 - (d) Document.
6. Trong Ví dụ 1 ở phần **Loading Documents**, PyPDFLoader tạo ra Document theo đơn vị nào?
- (a) Mỗi file PDF là một Document.
 - (b) Mỗi trang trong file PDF là một Document.
 - (c) Mỗi chunk là một Document.
 - (d) Mỗi câu là một Document.
7. Đối tượng nào biến các chunk thành vector embedding để lưu/tra cứu trong VectorStore?

```
1 docs = PyPDFLoader("sample.pdf").load()
2
3 splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=200)
4 chunks = splitter.split_documents(docs)
5
6 emb = OpenAIEmbeddings(model="text-embedding-3-large")
7 vs = Chroma.from_documents(chunks, embedding=emb)
8
9 retriever = vs.as_retriever(search_type="similarity", search_kwargs={"k": 4})
```

- (a) PyPDFLoader.
 - (b) RecursiveCharacterTextSplitter.
 - (c) OpenAIEmbeddings.
 - (d) Chroma (Vector Store).
8. Quan sát đoạn code cấu hình Text Splitter dưới đây:

```
1 class TextSplitter:
2     def init(self, chunk_size=400, chunk_overlap=120):
3         self.splitter = RecursiveCharacterTextSplitter(
```

```

4         separators=["\n\n", "\n", " ", ""],
5         chunk_size=chunk_size,
6         chunk_overlap=chunk_overlap,
7         length_function=len,
8     )

```

Trong bối cảnh xây dựng RAG, vai trò cụ thể của tham số `chunk_overlap=120` trong đoạn code trên là gì?

- (a) Chia văn bản thành các đoạn nhỏ có kích thước cố định đúng bằng 120 ký tự bất kể cấu trúc câu.
 - (b) Tự động loại bỏ 120 ký tự đặc biệt hoặc khoảng trắng thừa ở đầu mỗi đoạn văn.
 - (c) Duy trì sự liên tục về ngữ cảnh giữa các đoạn liền kề bằng cách lặp lại 120 ký tự cuối của chunk trước sang chunk sau.
 - (d) Giới hạn độ dài tối đa của mỗi chunk là 120 ký tự để giảm tải bộ nhớ khi embedding.
9. Khi khởi tạo pipeline sinh văn bản cho hệ thống RAG, đoạn code cấu hình thường được thiết lập như sau:

```

1 model_pipeline = pipeline(
2     "text-generation",
3     model=model,
4     tokenizer=tokenizer,
5     temperature=0.2,      # Lưu ý tham số này
6     max_new_tokens=450,
7     do_sample=True,
8     top_p=0.75
9 )

```

Trong ngữ cảnh của bài toán RAG, tại sao chiến lược đặt `temperature` ở mức thấp lại là tiêu chuẩn được khuyến nghị thay vì đặt ở mức cao?

- (a) Để ngăn chặn mô hình sinh ra các vòng lặp vô tận khi gặp các đoạn văn bản khó.
 - (b) Để tăng tính đa dạng cho câu trả lời, giúp mô hình có thể diễn đạt cùng một ý theo nhiều văn phong khác nhau trong mỗi lần chạy.
 - (c) Để giảm thiểu sự sáng tạo ngẫu nhiên, buộc mô hình tập trung vào các từ có xác suất cao nhất nhằm đảm bảo câu trả lời bám sát vào tài liệu được cung cấp.
 - (d) Để tăng tốc độ xử lý của mô hình, vì giá trị `temperature` cao đòi hỏi nhiều tài nguyên tính toán hơn để sinh ra các từ vựng phong phú.
10. Trước khi đưa các tài liệu tìm được vào Prompt, hàm helper `format_docs` thực hiện một bước lọc dữ liệu như sau:

```

1 def format_docs(docs):
2     formatted = []

```

```
3     seen = set()
4     for doc in docs:
5         content = doc.page_content.strip()
6         if content and len(content) > 40 and content not in seen:
7             formatted.append(content)
8             seen.add(content)
9     return "\n\n".join(formatted)
```

Mục đích thực tế của điều kiện `len(content) > 40` và `content not in seen` trong đoạn code trên là gì?

- (a) Đảm bảo chỉ có tối đa 40 tài liệu quan trọng nhất được gửi vào mô hình để xử lý.
- (b) Loại bỏ các đoạn văn bản quá ngắn, thiếu ngữ nghĩa và ngăn chặn việc lặp lại thông tin để tối ưu hóa chất lượng Prompt.
- (c) Kiểm tra xem tài liệu có chứa ít nhất 40 từ khóa quan trọng liên quan đến câu hỏi hay không.
- (d) Giới hạn tổng độ dài của prompt không vượt quá 40 token để tiết kiệm chi phí API và tránh lỗi Context Window.

Phụ lục

1. **Solution:** File code cài đặt thực nghiệm có thể được tải tại [đây](#).
2. **Q&A:** Bạn có thể đặt thêm câu hỏi về nội dung bài đọc trong group Facebook hỏi đáp tại [đây](#). Tất cả câu hỏi sẽ được trả lời trong vòng tối đa 4 tiếng.

AIO_QAs-Verified

🔒 Private group · 1.4K members



Hình 12: Hình ảnh group facebook AIO Q&A.